Create a Microsoft Foundry project

Deploy a model

Chat with the model

View client code to chat with a model

Specify instructions in a system prompt

Save the model configuration as an agent

Add a knowledge tool to the agent

View client code to access the agent in your project

Publish the agent and use it in a client app

Summary

Clean Up

Get started with generative AI and agents in Microsoft Foundry

In this exercise, you'll use Microsoft Foundry to deploy and explore a generative AI model. You'll then use the model in an agent that includes knowledge tools to answer user questions.

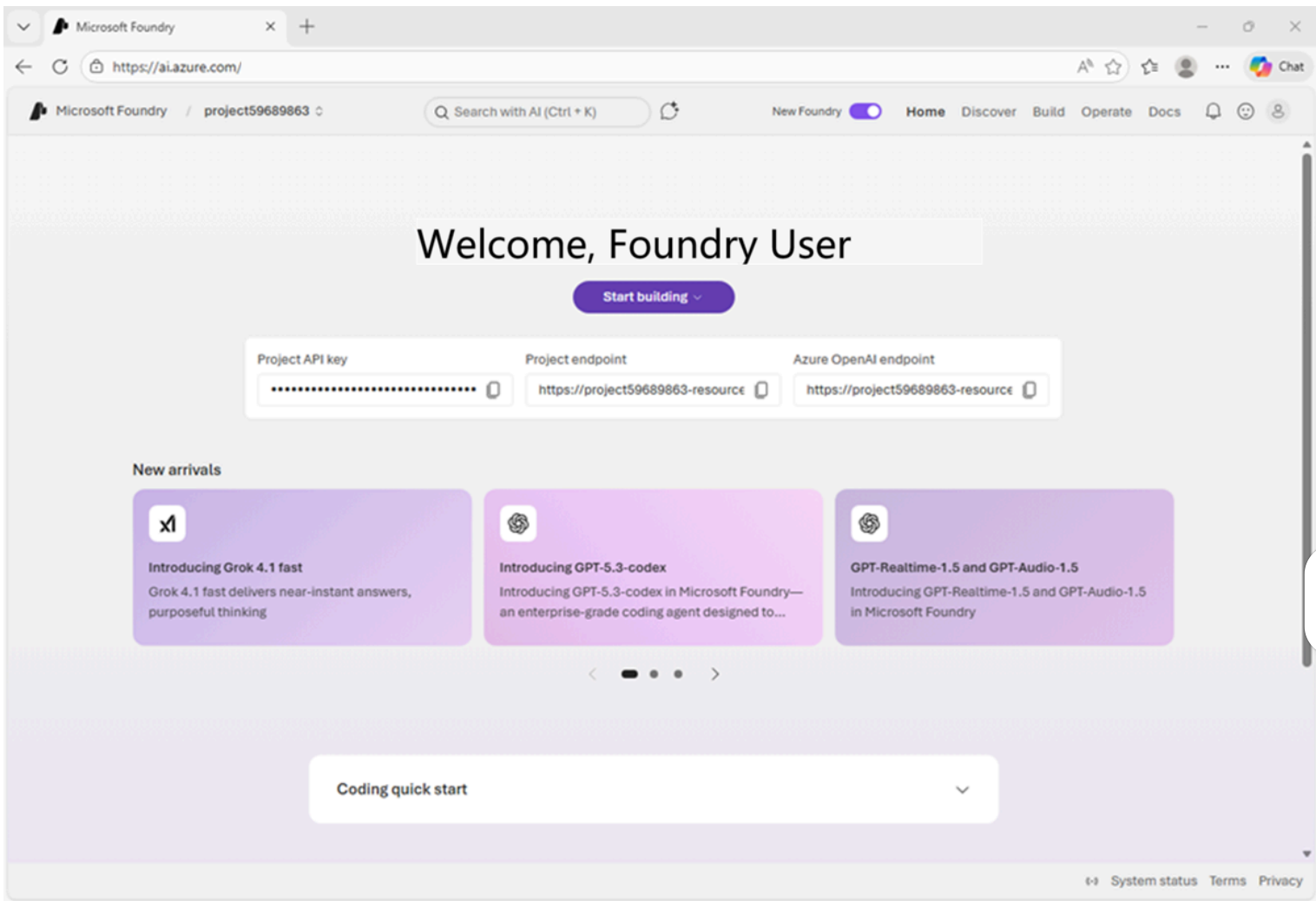
> **Note**: Many components of Microsoft Foundry, including the Microsoft Foundry portal, are subject to continual development. This reflects the fast-moving nature of artificial intelligence technology. Some elements of your user experience may differ from the images and descriptions in this exercise!

This exercise should take approximately **35** minutes to complete.

Create a Microsoft Foundry project

Microsoft Foundry uses *projects* to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open Microsoft Foundry at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the **Foundry** logo at the top left to navigate to the home page.
2. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
   - **Foundry resource**: *Enter a valid name for your AI Foundry resource.*
   - **Subscription**: *Your Azure subscription*
   - **Resource group**: *Create or select a resource group*
   - **Region**: Select any of the **AI Foundry recommended** regions
3. Select **Create**. Wait for your project to be created. It may take a few minutes. After creating or selecting a project in the new Foundry portal, it should open in a page similar to the following image:

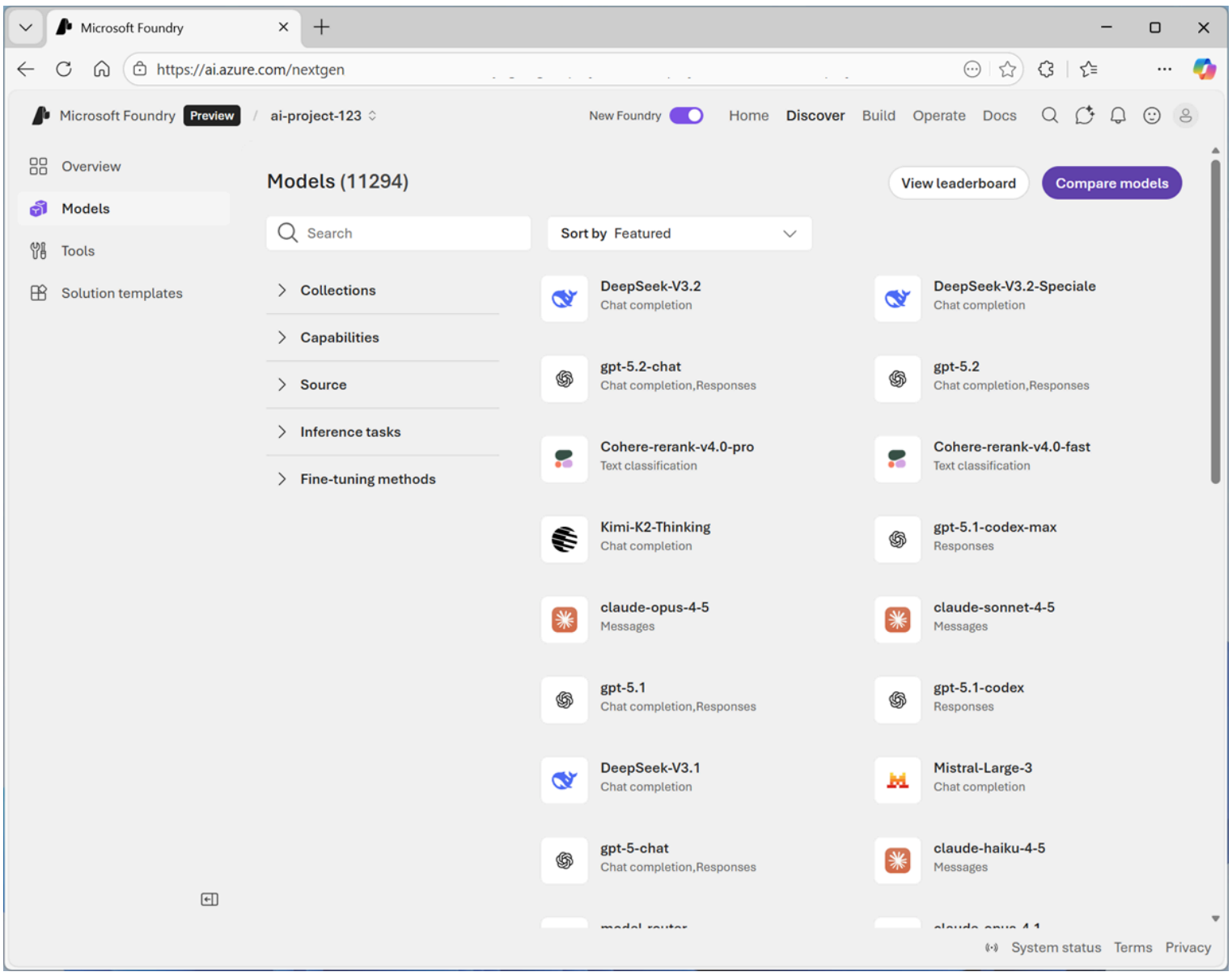


Deploy a model

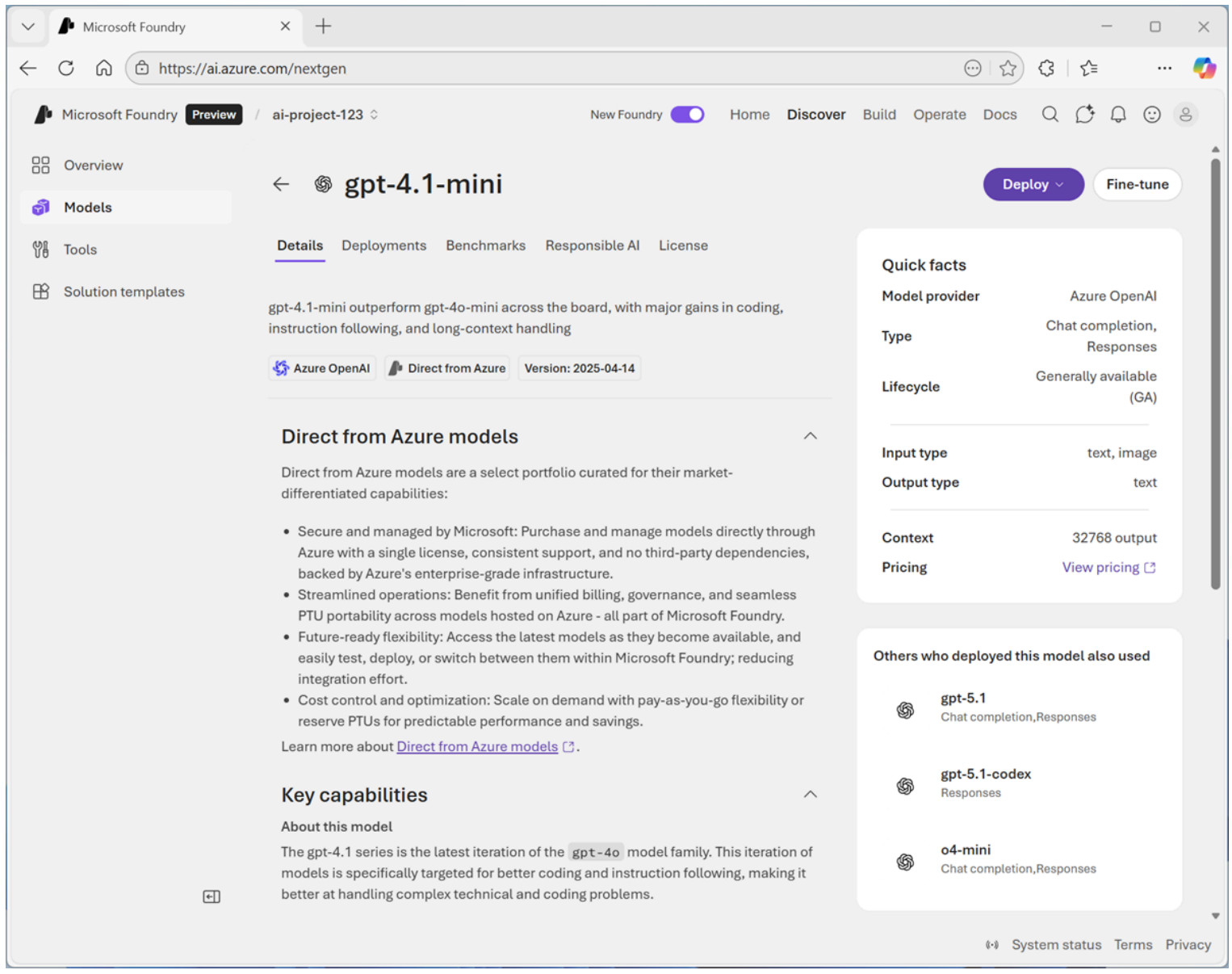
At the heart of every generative AI app or agent, there's a language model - usually a large language model (LLM), though in some cases a more compact small language model (SLM) may be used.

1. Now you're ready to **Start building**. Select **Find models** (or on the **Discover** page, select the **Models** tab) to view the Microsoft Foundry model catalog.

   Microsoft Foundry provides a large collection of models from Microsoft, OpenAI, and other providers, that you can use in your AI apps and agents.



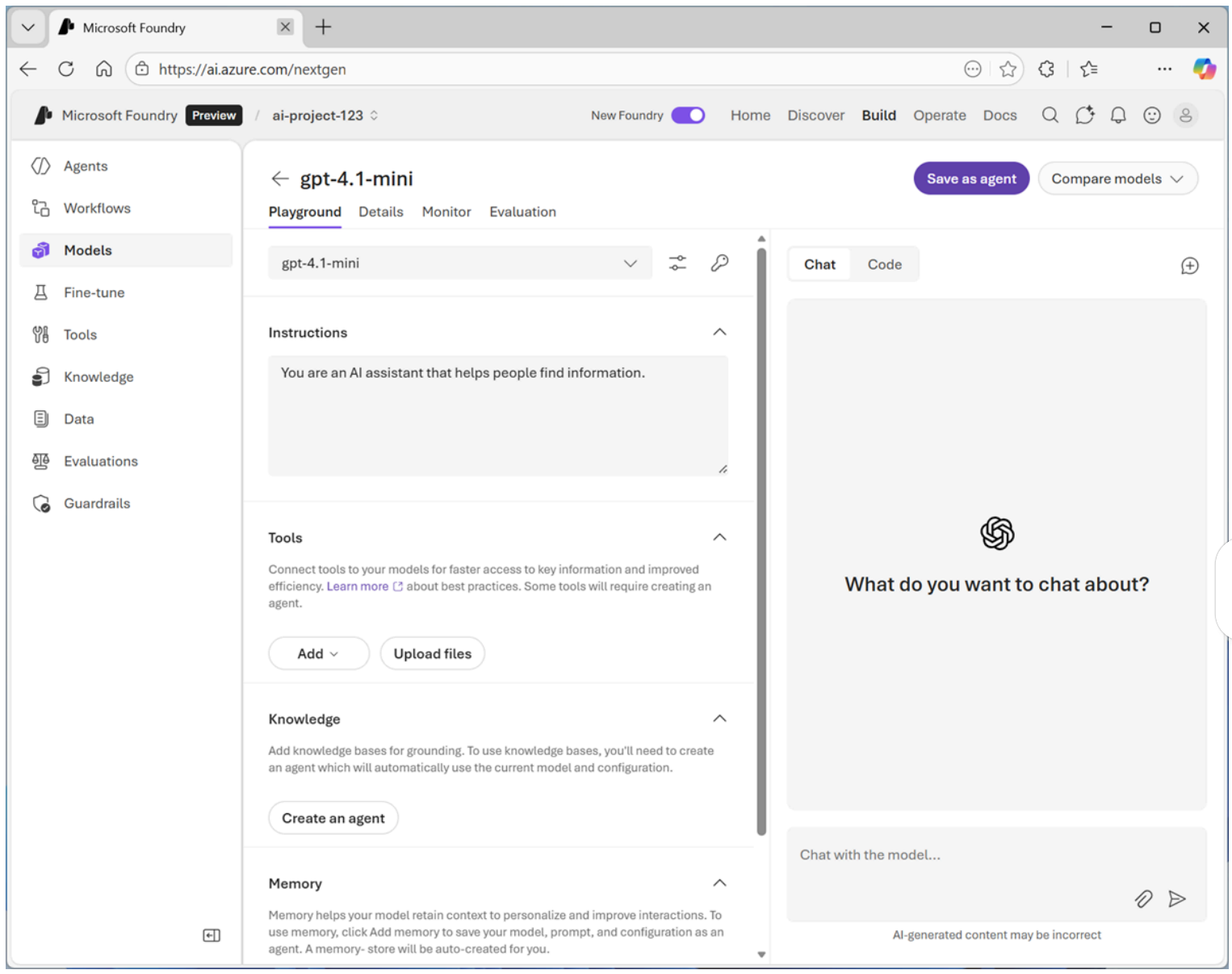
2. Search for and select the `gpt-4.1-mini` model, and view the page for this model, which describes its features and capabilities.



3. Use the **Deploy** button to deploy the model using the default settings. Deployment may take a minute or so.

> **Tip**: Model deployments are subject to regional quotas. If you don't have enough quota to deploy the model in your project's region, you can use a different model - such as gpt-4.1-nano, or gpt-4o-mini. Alternatively, you can create a new project in a different region.

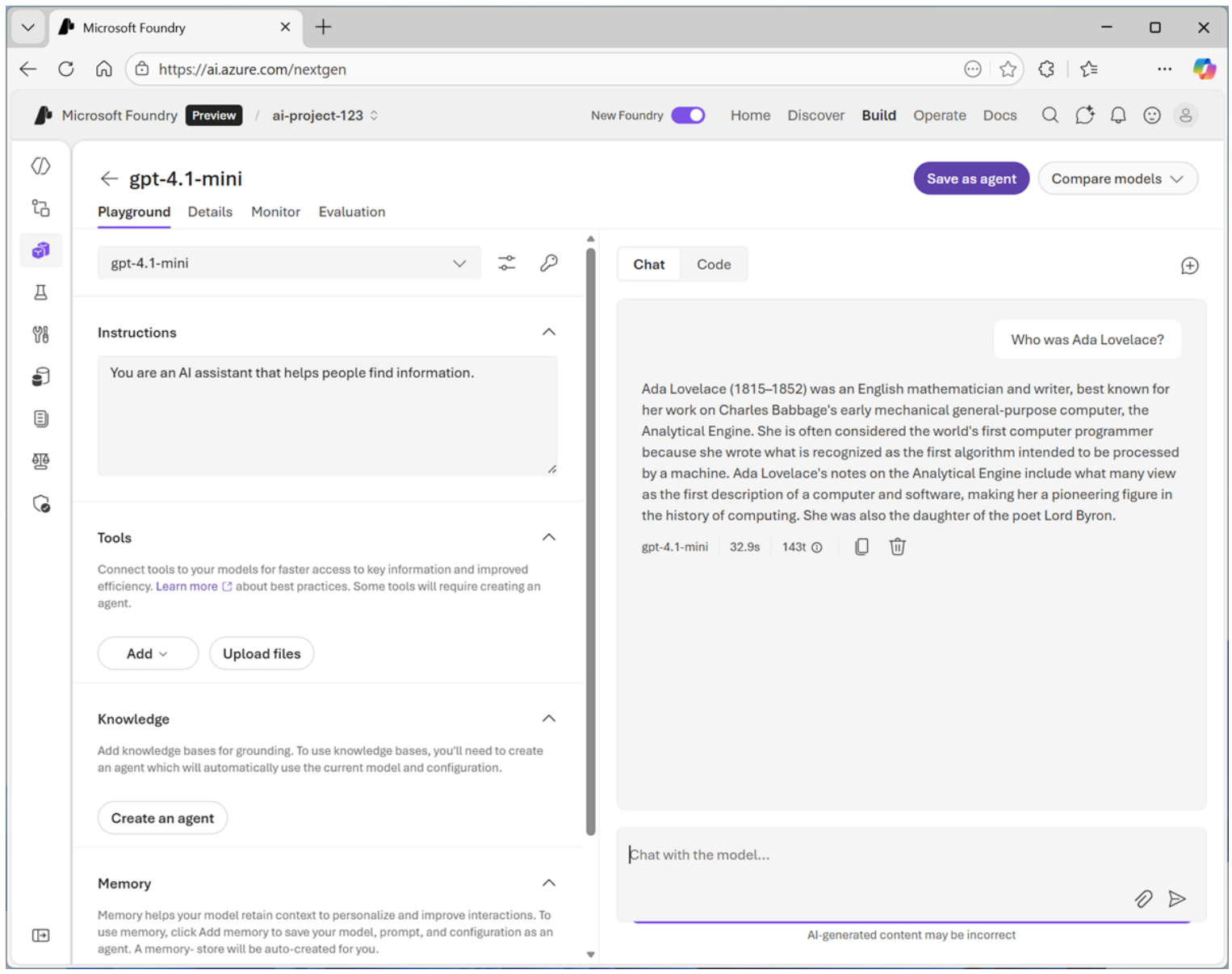
4. When the model has been deployed, view the model playground page that is opened, in which you can chat with the model.



Chat with the model

You can use the playground to explore the model by chatting with it and observing the effect of changes to settings like its instructions (sometimes called the *system prompt*) and parameters.

1. Use the button at the bottom of the left navigation pane to hide it and give yourself more room to work with.
2. In the **Chat** pane, enter a prompt such as `Who was Ada Lovelace?`, and review the response.



3. Enter a follow-up prompt, such as `Tell me more about her work with Charles Babbage.` and review the response.

> **Note**: Generative AI chat applications often include the conversation history in the prompt; so the context of the conversation is retained between messages. In this case, "her" is interpreted as referring to Ada Lovelace.

4. At the top-right of the chat pane, use the **New chat** button to restart the conversation. This removes all conversation history.
5. Enter a new prompt, such as `Tell me about the ELIZA chatbot.` and view the response.
6. Continue the conversation with prompts such as `How does it compare with modern LLMs?`.

View client code to chat with a model

When you're satisfied with the responses a model returns in the playground, you can develop client applications that consume it. Microsoft Foundry provides a REST API and multiple language-specific SDKs that you can use to connect to the deployed model and chat with it.

1. In the **Chat** pane, view the **Code** tab. This tab shows sample code that a client application can use to chat with the model. Above the sample code, you can choose preferences for:
   - **API**: The OpenAI API is a common standard for implementing conversations with generative AI models. There are two variants of the OpenAI API that you can use:
     - **Completions**: A broadly used programmatic syntax for submitting prompts to a model.

        - **Responses**: A newer syntax that offers greater flexibility for building apps that converse with both standalone models and with *agents*.
    - **Language**: You can write code to consume a model in a wide range of programming languages, including Python. Microsoft C#, JavaScript, and others.
    - **SDK**: You can use a language-specific SDK, which encapsulates the low-level communication details between the client and model; or you can work directly with the REST API, enabling you to have full control over the HTTP request messages that your client sends to the model.
    - **Authentication**: To use a model deployed in Microsoft Foundry, the client application must be authenticated. You can implement authentication using:
        - **Key-based authentication**: The client app must present a security key (which you can find by selecting the key icon above the code sample)
        - **Microsoft Entra ID authentication**: The client app presents an authentication token based on an identify that is assigned to it (or to the current user).
2. Select the following code options:
    - **API**: Responses API
    - **Language**: Python
    - **SDK**: OpenAI SDK
    - **Authentication**: Key authentication

    The resulting sample should be similar to the following code:

    Code Copy

```python
from openai import OpenAI

endpoint = "https://{your-foundry-resource}.openai.azure.com/openai/v1/"
deployment_name = "gpt-4.1-mini"
api_key = "<your-api-key>"

client = OpenAI(
    base_url=endpoint,
    api_key=api_key
)

response = client.responses.create(
    model=deployment_name,
    input="What is the capital of France?",
)

print(f"answer: {response.output[0]}")
```

    The code connects to the **OpenAI** endpoint for your Microsoft Foundry resource, using its secret authentication key (which you would need to copy into the code to set the **api_key** variable). It then uses the **responses.create** method to generate a response from your deployed model from an input prompt (in this case, the hard-coded question "What is the capital of France?") and prints the response to the output console.

Specify instructions in a *system prompt*

So far, you've used the model to provide general information. To support specific use cases, you should use a *system prompt* to provide the model with instructions that guide its responses. You can use the system prompt to give the model a specific focus or role, and provide guidelines about format, style, and constraints about what the model should and should not include in its responses.

For example, suppose an organization wants to use a generative AI model to power an AI agent that assists employees with expense claims.

1. In the model playground, switch back to the **Chat** tab. Then, at the top-right of the chat pane, use the **New chat** button to restart the conversation and removes the conversation history.

2. In the pane on the left, in the **Instructions** text area, change the system prompt to:

   Code Copy

   ```
   You are a helpful AI assistant who supports employees with expense claims. Provide concise, accurate information only on topics related to expenses. Do not provide any information about topics that are not directly related to expenses.
   ```

3. Now enter a new user prompt related to expense claims, such as `What kinds of business expense are typically reimbursed by employers?`

   Review the response, which should provide some general guidance about expense claims.

4. Try re-asking a previously-asked question that is unrelated to expenses, such as `Tell me about the ELIZA chatbot`; and compare the response now that the system prompt has changed.

   So far, we've specified instructions in the *playground*; but they're not saved outside of that environment. In a client application, you would need to include the system prompt as an **instructions** parameter in the **responses.create** method, like this:

   Code Copy

   ```
   response = client.responses.create(
           model=deployment_name,
           instructions="""
               You are a helpful AI assistant who supports employees with expense claims.
               Provide concise, accurate information only on topics related to expenses.
               Do not provide any information about topics that are not directly related to
   expenses.
           """
           input="What kinds of business expense are typically reimbursed by employers?",
       )
   ```

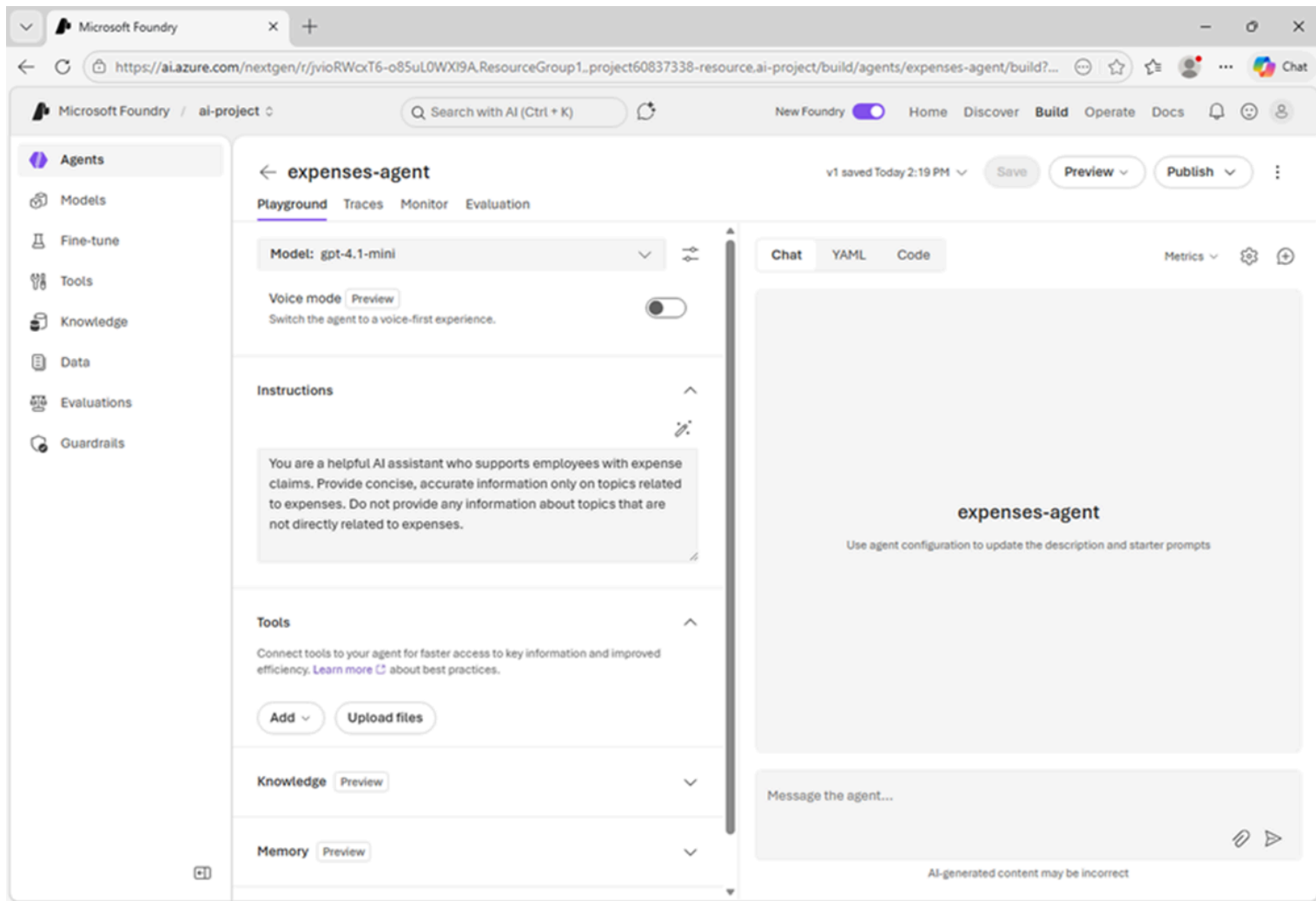
   To encapsulate the instructions and model in a single AI entity, we need to save the configuration as an *agent*.

Save the model configuration as an agent

While you can implement generative AI apps using a standalone model, to create a fully agentic AI experience, you need to encapsulate the model, its instructions, and any tool configuration that provides additional functionality, in an *agent*.

1. In the model playground, at the top right select **Save as agent**. Then, when prompted, name your new agent `expenses-agent`.

   When the agent is created, it opens in a new playground specifically for working with agents.



2. In the pane on the right, view the **YAML** tab, which contains the definition for your agent. Note that its definition includes the model, its parameter settings, and the instructions you specified - similar to this:

Code Copy

```
metadata:
  logo: Avatar_Default.svg
  microsoft.voice-live.enabled: "false"
object: agent.version
id: expenses-agent:1
name: expenses-agent
version: "1"
description: ""
created_at: 1776115196
definition:
  kind: prompt
  model: gpt-4.1-mini
  instructions: You are a helpful AI assistant who supports employees with expense claims.
Provide concise, accurate information only on topics related to expenses. Do not provide any
information about topics that are not directly related to expenses.
  temperature: 1
  top_p: 1
  tools: []
 status: active
```

3. Switch back to the **Chat** tab, and enter the prompt `Who are you?`

   The response should indicate that the agent is “aware” of its role as an expense claims advisor.

4. Enter an expenses-related prompt, such as `How much can I claim for a taxi?`

   The response is likely to be generic. Accurate; but not particularly helpful to the employee. We need to give the agent some knowledge about the company’s expense policies and procedures.

Add a knowledge tool to the agent

Agents use *tools* to perform tasks or find information. You can use a general web search tool or a simple file search tool to provide a source of knowledge; or for more comprehensive agentic solutions, you can create a *Microsoft Foundry IQ* knowledge store that connects the agent to one or more data sources within your enterprise. In this exercise, we'll use a simple file search tool.

1. Open a new browser tab, and view the **expenses_policy.docx** at `https://microsoftlearning.github.io/mslearn-ai-fundamentals/data/expenses_policy.docx`. We'll use this to provide a knowledge source that the agent can use to answer questions about expense claims.
2. Download **expenses_policy.docx** to your local computer.
3. Return to the tab containing the agent playground, and in the pane on the left, expand the **Tools** section if it's not already expanded.
4. Upload the **expenses_policy.docx** file, creating a new index with the default index name. When the index has been created, attach it to the agent.
5. At the top of the agent playground, use the **Save** button to update the agent definition.
6. In the pane on the right, view the **YAML** tab, which contains the definition for your agent. Note that its definition now includes the file search tool you added (in the **tools** section):

   Code Copy

   ```
   metadata:
     logo: Avatar_Default.svg
     description: ""
     modified_at: "1776115781"
     microsoft.voice-live.enabled: "false"
   object: agent.version
   id: expenses-agent:2
   name: expenses-agent
   version: "2"
   description: ""
   created_at: 1776115782
   definition:
     kind: prompt
     model: gpt-4.1-mini
     instructions: You are a helpful AI assistant who supports employees with expense claims.
   Provide concise, accurate information only on topics related to expenses. Do not provide any
   information about topics that are not directly related to expenses.
     temperature: 1
     top_p: 1
     tools:
       - type: file_search
         vector_store_ids:
           - vs_tmwFZKmfVB3rZJoeaJAcgdy9
   status: active
   ```

7. Switch back to the **Chat** tab, and enter the same expenses-related prompt as before (for example, `How much can I claim for a taxi?`) and view the response.

   This time the response should be informed by the information in the expenses data source.

8. Try a few more expenses-related prompts, like `What about a hotel?` or `Can I claim the cost of my dinner?`

   Congratulations! We have a working agent with access to the knowledge it needs. Now we're ready to develop apps that use it.

View client code to access the agent in your project

The agent is defined within your Foundry project, and there's a convenient way to develop apps that connect to it there; allowing you to iteratively refine both the agent and the client app to create the solution you need.

1. In the agent playground, switch from the **Chat** tab to the **Code** tab, and view the sample code for consuming the agent; which should be similar to this:

   Code Copy

   ```
   # Before running the sample:
   # pip install azure-ai-projects>=2.0.0

   from azure.identity import DefaultAzureCredential
   from azure.ai.projects import AIProjectClient

   my_endpoint = "https://{your-foundry-resource}}.services.ai.azure.com/api/projects/{your-project}"

   project_client = AIProjectClient(
       endpoint=my_endpoint,
       credential=DefaultAzureCredential(),
   )

   my_agent = "expenses-agent"
   my_version = "2"

   openai_client = project_client.get_openai_client()

   # Reference the agent to get a response

   response = openai_client.responses.create(
       input=[{"role": "user", "content": "Tell me what you can help with."}],
       extra_body={"agent_reference": {"name": my_agent, "version": my_version, "type": "agent_reference"}},
   )

   print(f"Response output: {response.output_text}")
   ```

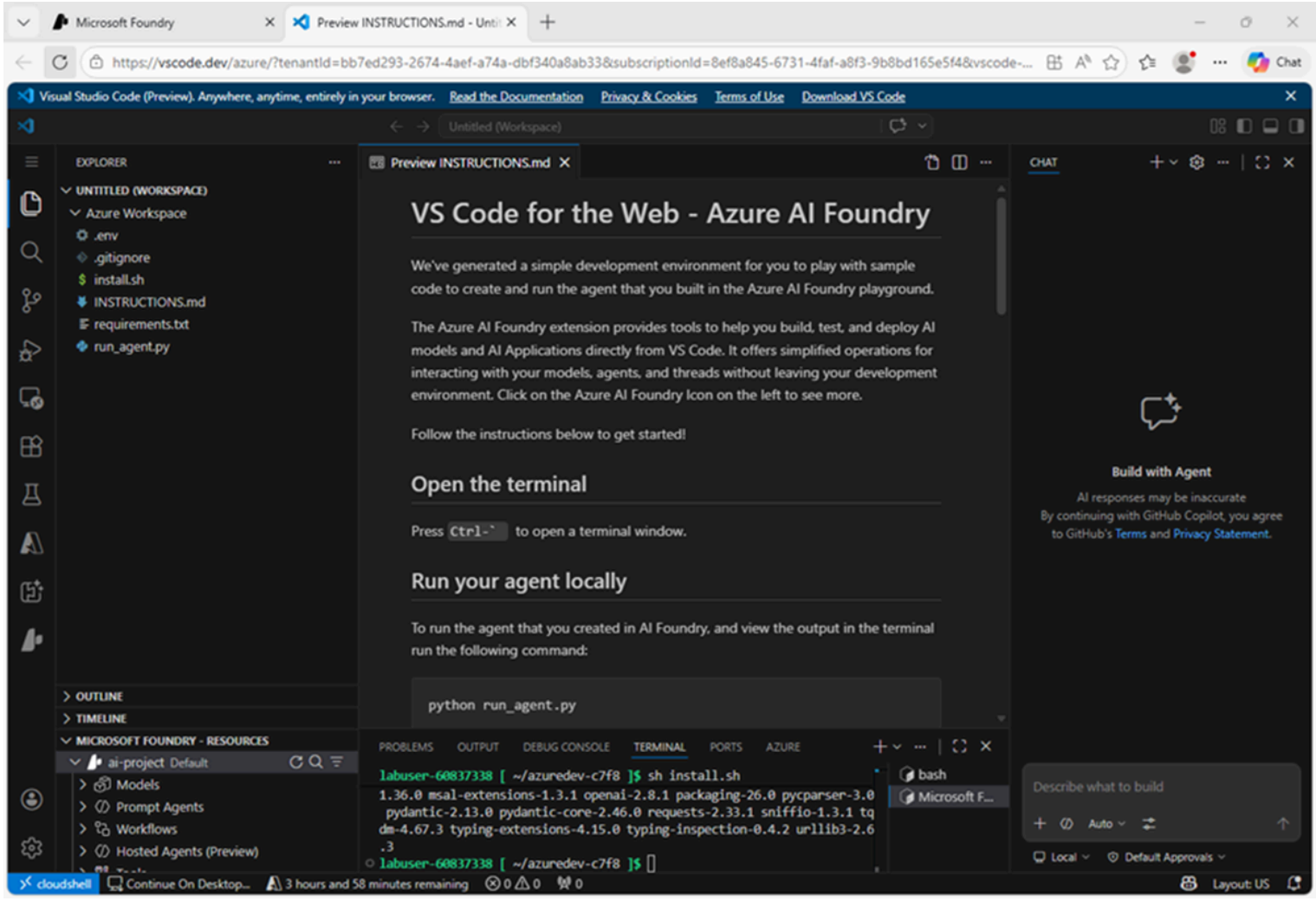
   The code to connect to your agent uses the **Azure.AI.Projects** library to create an **AIProjectClient** object connected to your Foundry project. Since this involves connecting to a project, which may contain priveleged resources, key-based authentication is not supported, and the application must use an Entra ID identity to be authenticated.

   After connecting to the project, the code uses the project client's **get_openai_client** method to retrieve an OpenAI client object; with which it can submit prompts to the agent using the same **Responses** API we peviously saw being used to chat with a model. Since a project can contain multiple agents and models, the specific agent details are specified as **extra_body** in the **responses.create** method.

2. In the **Code** tab, use the **Open in VS Code for the web** button to open Visual Studio Code for the Web in a new browser tab.

   Wait for the environment to be set up.

   > **Tip**: It can take a few minutes to set the envionment up!



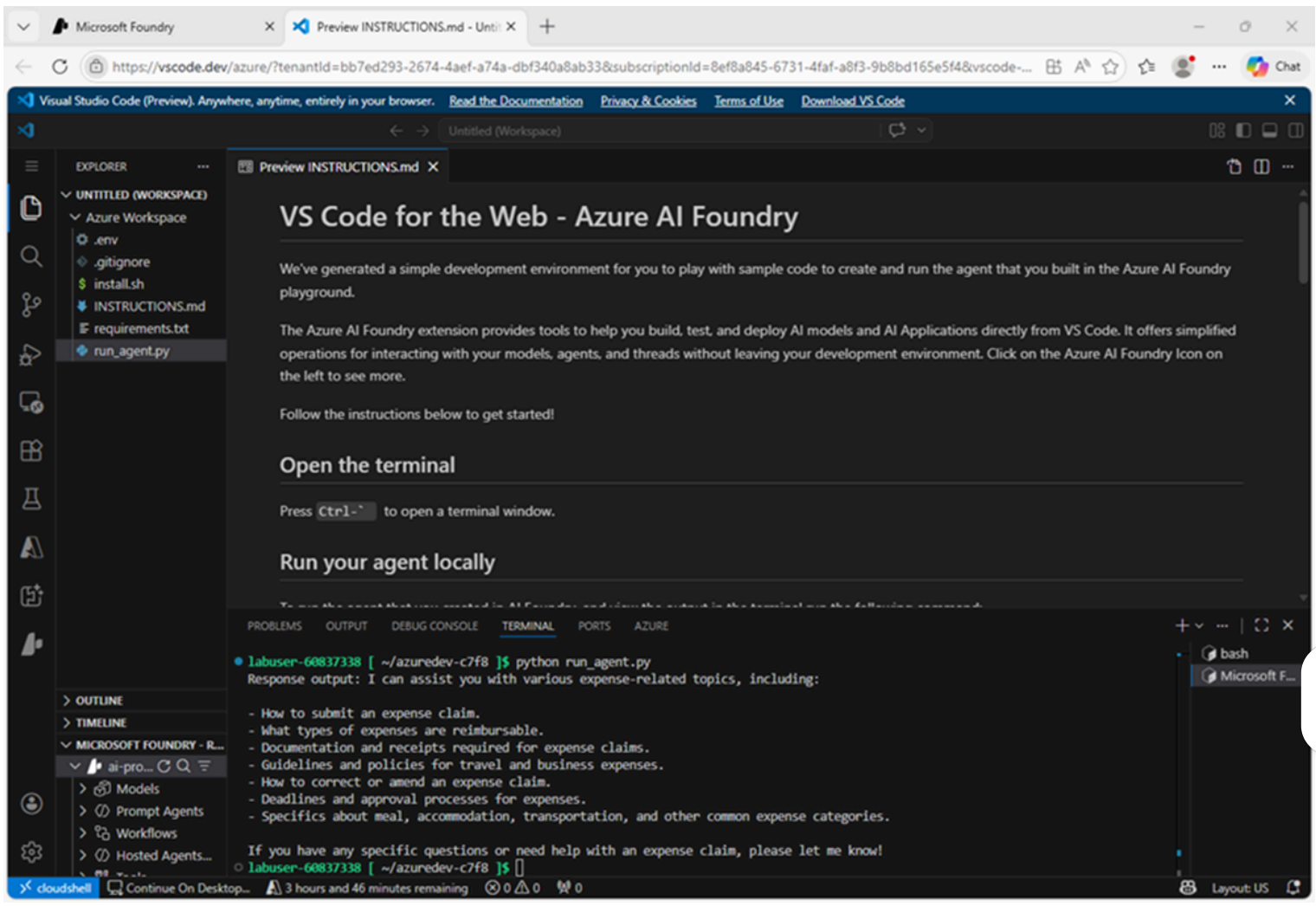
3. After VS Code for the web has opened and the environment has been set up, close the GitHub Copilot **Chat** pane on the right side to give you more room, and note that the **Instructions.md** file contains the instructions you need to run the sample code (which is in the **run_agent.py** file in the VS Code Explorer pane on the left.)

4. In the terminal pane at the bottom, enter the following command to run the code:

Code Copy

```
python run_agent.py
```

The output should inlude a response to the prompt *Tell me what you can help with.*



> **Tip**: If an authentication issue occurs, you may need to sign into Azure in the VS Code terminal by using the Azure CLI `az login` command. See the Azure CLI documentation for details.

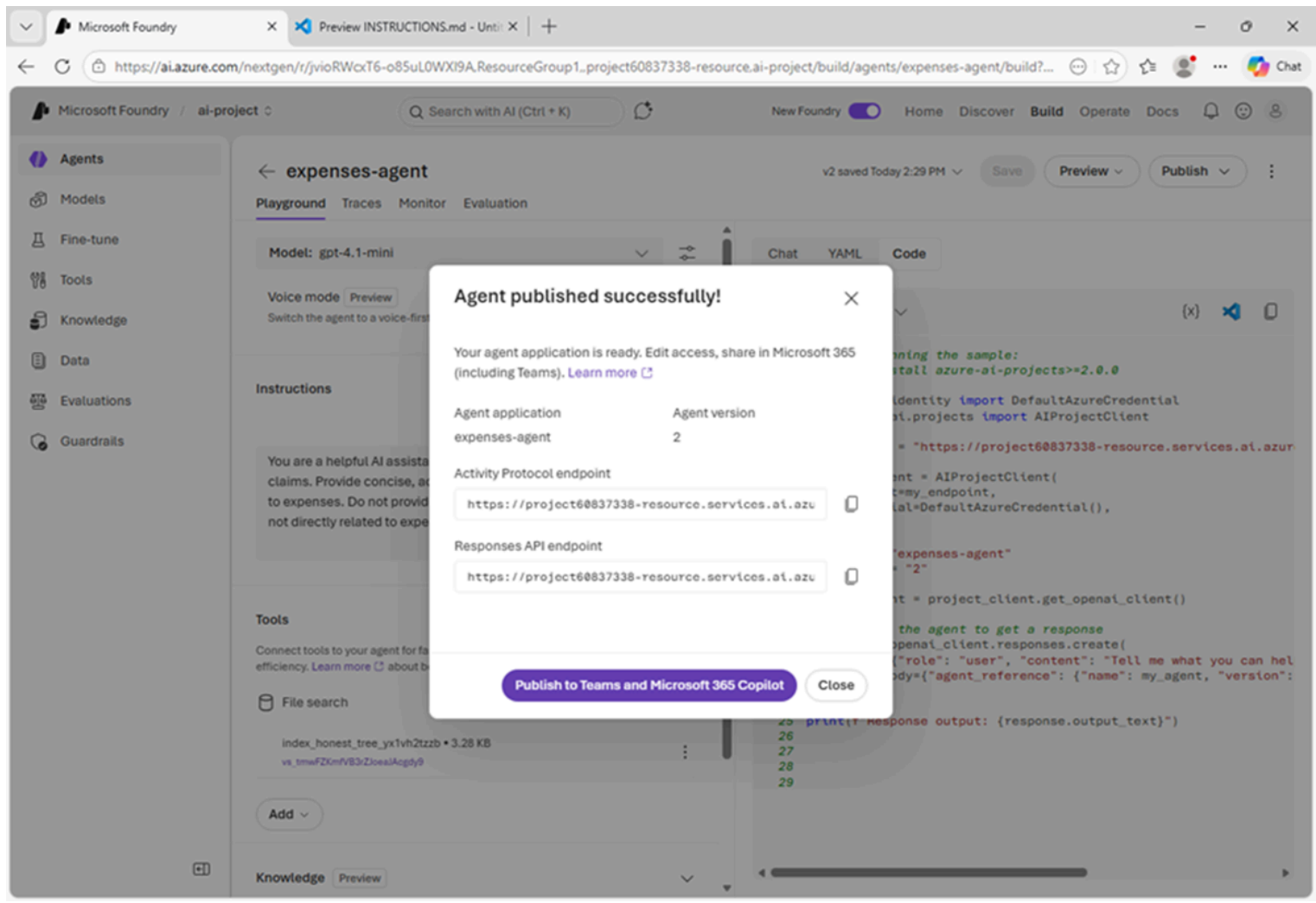
Publish the agent and use it in a client app

When you're satisfied with your agentic solution, you can *publish* the agent to its own dedicated endpoint, and adapt client applications to use it from there. Publishing the agent makes it available independently from the project, making it a more suitable way to deploy the agent in production.

1. Keep the VS Code for the Web tab open, but switch back to the Foundry portal tab.

2. In the agent playground, in the **Publish** drop-down list, select **Publish agent**.

   When prompted, confirm you want to publish the agent to production, and after a few seconds, view the published agent details. In particular, note the Responses API endpoint that clients apps can use to connect to your agent.



3. Note that you can perform additional steps to publish your agent for integration with Teams and Microsoft 365 Copilot. However, in this exercise, select **Close**.

   > **Tip**: You can use the **View details** option in the **Publish** drop-down list to re-open the agent details.

4. Switch back to the VS Code for the Web tab, and in the Explorer pane, add a new file named `expenses-client.py`.

5. Add the following code to the new **expenses-client.py** file.

   Code Copy

```
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

# Replace with your agent endpoint
AGENT_ENDPOINT = "YOUR_AGENT_ENDPOINT"

# Create OpenAI client authenticated with Azure credentials
openai = OpenAI(
     api_key=get_bearer_token_provider(DefaultAzureCredential(),
"https://ai.azure.com/.default"),
     base_url=AGENT_ENDPOINT,
     default_query={"api-version": "2025-11-15-preview"}
)

# Send a request to the published agent
response = openai.responses.create(
     input=input("Prompt:\n"),
)
print(f"Response output:\n{response.output_text}")
```

This code uses the **Open AI Responses** API with Entra ID authentication. Since the agent is published in its own production endpoint, there's no need to connect to the Foundry project using the **Azure.AI.Projects** library or to specify agent details in the **responses.create** method call.

6. Replace the **YOUR_AGENT_ENDPOINT** placeholder with the Responses API endpoint for your agent (copied from the published agent details in the Foundry portal).
7. Save the changes to the **expenses-client.py** code file (CTRL+S).
8. In the VS Code terminal pane, enter the following command to run the code.

Code Copy

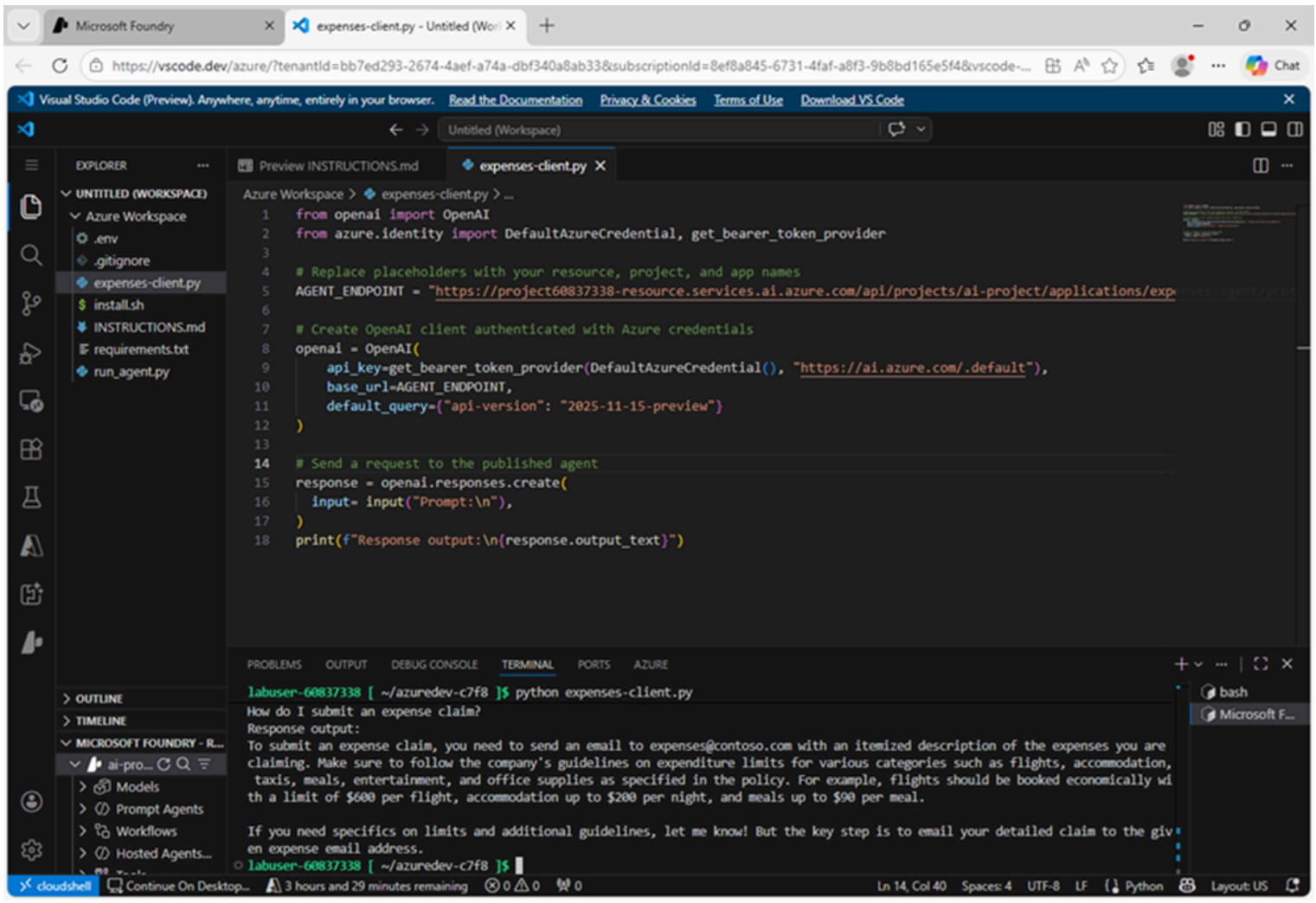
```
python expenses-client.py
```

9. When prompted, enter the following prompt:

Code Copy

```
How do I submit an expense claim?
```

The code uses our published agent to get a response, and displays it.



Summary

In this exercise, you explored how to deploy and chat with a generative AI model in Microsoft Foundry portal. You then saved the model as an agent, and configured the agent with instructions and tools before exploring options for deploying and using the agent.

The agent explored in this exercise is a simple example that demonstrates how quickly and easily you can get started with generative AI app and agent development using Microsoft Foundry. From this foundation, you could build a comprehensive agentic solution in which agents use tools to find information and automate tasks, and collaborate with one another to perform complex workflows.

Clean Up

If you have finished exploring Microsoft Foundry, you should delete the resources created in this exercises to avoid unnecessary utilization charges.

1. Open the Azure portal at `https://portal.azure.com` and view the contents of the resource group where you deployed the project used in this exercise.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.

> **Tip**: If you want to keep the Foundry project, but avoid being charged for the published agent, use the ⋮ menu next to the **Publish** drop-down list to delete the agent.